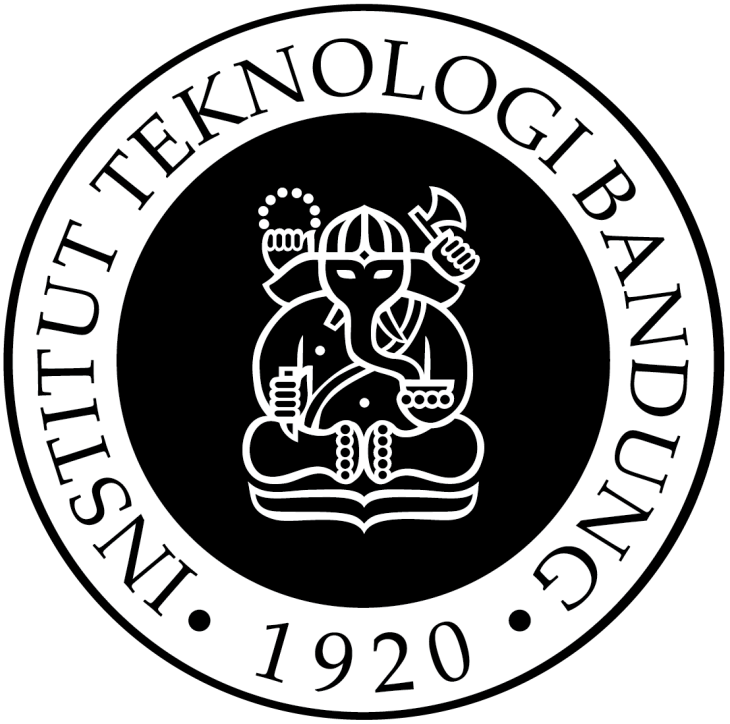
Laporan Perancangan RTL Convolution, Maxpooling, dan ReLU



Aziiz Adityatama
NIM 13224070

Teknik Elektro
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
18 Oktober 2026

Bab I : Spesifikasi & Alur Pemrosesan Sistem

1.1. Spesifikasi Sistem

Perancangan akselerator RTL untuk CNN, konvolusi dengan 3 kernel 3x3, maxpooling 2x2, dan ReLU dari input 16x16 menjadi 7x7x3 bit 8.

Bagian	Spesifikasi
Input	16x16, 8 bit unsigned, 1 channel
Kernel	3 buah, 3x3, 8 bit signed
Konvolusi	Stride 1, tanpa padding, output 14x14 per kernel
Jumlah channel	3 buah, satu per kernel
Maxpooling	2x2, stride 2
ReLU	$f(x) = x$ jika $x \geq 0$, 0 jika$x < 0$
Output	7x7x3, 8 bit unsigned

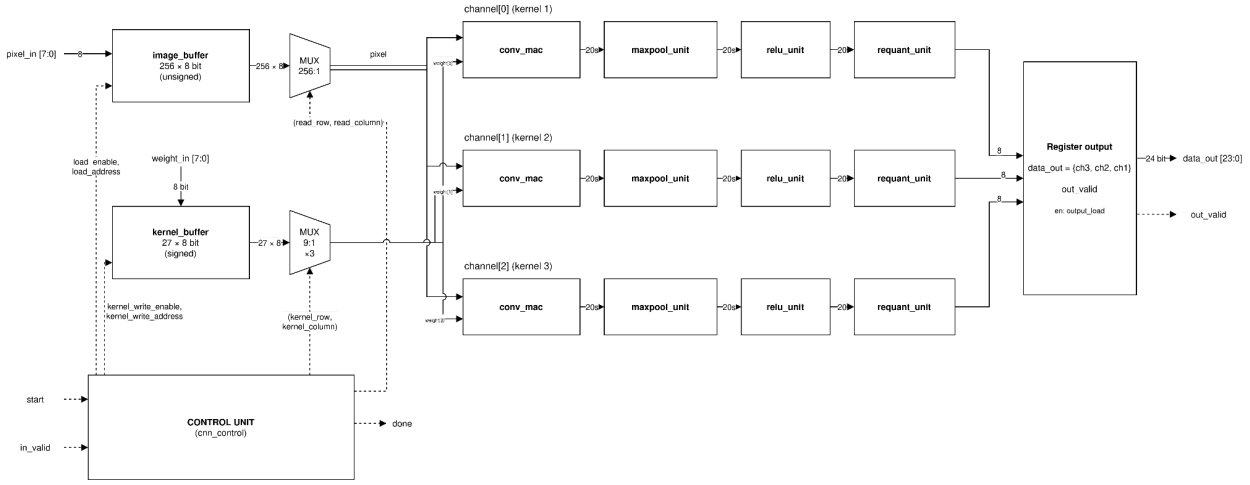
1.2. Alur Pemrosesan

Akselerator ini memiliki alur pemrosesan:

1. Konvolusi
   Untuk setiap kernel dan posisi (i, j), sembilan pixel 3x3 dikalikan dengan kernel lalu dijumlahkan. Ketiga kernel diterapkan pada input yang sama.
2. Maxpooling
   Setiap feature map 14x14 dibagi menjadi 2x2, nilai terbesar diambil sehingga ukurannya menjadi 7x7
3. ReLU
   Nilai negatif diganti menjadi 0 dan nilai selain itu bernilai tetap.
4. Requantization
   Hasil ReLU diskalakan ke 8 bit karena spesifikasi output 8 bit unsigned

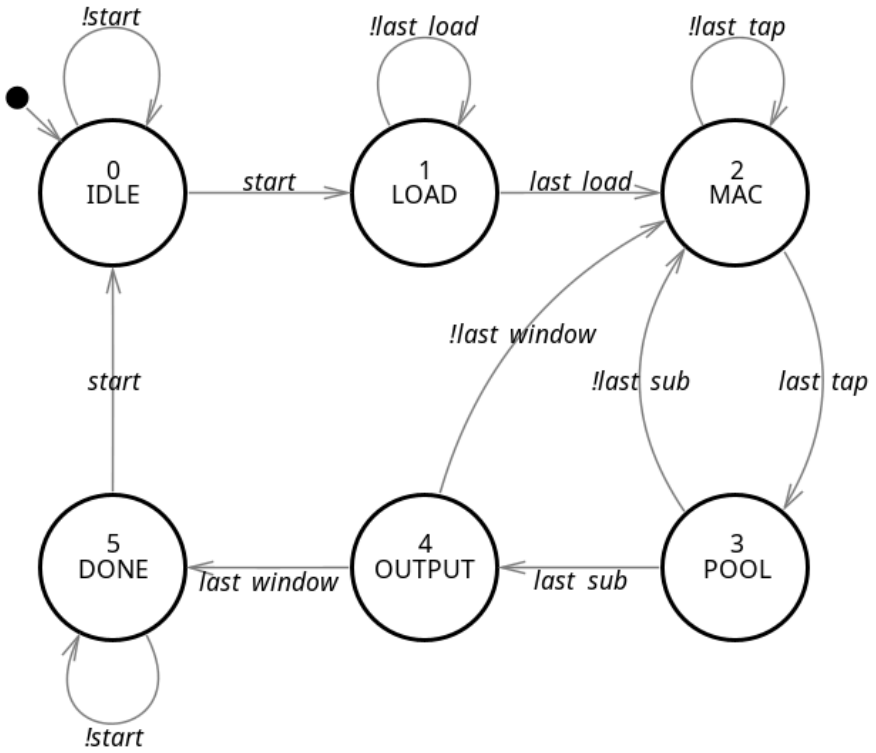
Bab II : Desain

Desain menggunakan arsitektur FSMD dengan tiga unit MAC paralel, satu per kernel, dan menghitung konvolusi dalam urutan jendela pooling sehingga tidak dibutuhkan buffer hasil konvolusi.



Gambar 2.1 Datapath Arsitektur

Modul	Fungsi
cnn_top	Top level, port, parameter, register output
cnn_control	FSM 6 state, counter, alamat baca/tulis
image_buffer	Buffer 256 x 8 bit
kernel_buffer	Buffer kernel
conv_mac	Perkalian 8 bit unsigned x 8 bit signed + akumulator 20 bit
maxpool_unit	Register maksimum sementara + komparator
relu_unit	ReLU
requant_unit	Requantization ke 8 bit



Gambar 2.2 Diagram state FSM

State	Durasi	Fungsi
IDLE	-	Menunggu start, mengosongkan counter
LOAD	256 clock	Menulis pixel, dan nilai kernel di 27 clock pertama
MAC	9 clock	Ketiga MAC mengakumulasi satu tap per clock
POOL	1 clock	Hasil konvolusi, register maksimum diperbarui, akumulator dikosongkan
OUTPUT	1 clock	ReLU dan requantization dari nilai maksimum, dimasukkan ke register output
DONE	-	Menunggu start berikutnya

Bab III : Implementasi RTL

RTL ditulis dalam verilog, satu modul per file dengan parameterisasi.

Parameter	Nilai	Keterangan
c	16	Ukuran input

KERNEL_SIZE	3	Ukuran kernel
NUM_KERNELS	3	Jumlah kernel dan channel
POOL_WINDOW	2	Ukuran dan stride pooling
DATA_WIDTH	8	Lebar pixel, weight, dan output
ACC_WIDTH	20	Lebar akumulator
SHIFT	8	Requantization

Bab IV : Simulasi

Simulasi RTL dilakukan dengan mencocokkan dengan nilai golden model pada keempat tahap, konvolusi, pooling, ReLU, dan output. Golden model ditulis menggunakan scripts python, hasil setiap tahap diekspor ke file hex sebagai nilai referensi.

Bagian	Keterangan
Cek Konvolusi	Saat pool_update = 1, akumulator ketiga channel dibandingkan dengan nilai referensi
Cek pooling dan ReLU	Saat output_load = 1, register maksimum dan keluaran ReLU dibandingkan
Cek Output	Saat out_valid = 1, data_out dibandingkan
Cetak Matriks	Cetak matriks input, kernel, dan output per tahap
Ringkasan	Jumlah nilai diperiksa dan salah per tahap, total clock

4.1. Hasil Simulasi

Berikut merupakan hasil simulasi dengan testbench

```
Tahap        | Cek | Salah
-------------+-----+------
Konvolusi    | 588 |   0
Maxpooling   | 147 |   0
ReLU         | 147 |   0
Output 8-bit | 147 |   0

Total clock : 2265
HASIL: SEMUA LULUS
tb/tb_cnn_top.v:240: $finish called at 22690000 (1ps)
```

Gambar 4.1 Screenshot terminal ringkasan make sim

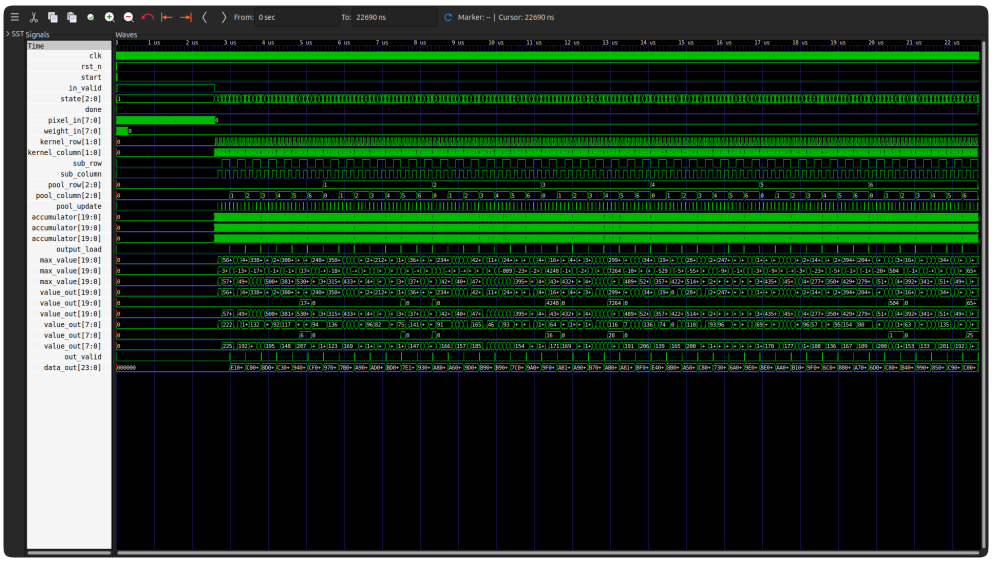
Tahap	Nilai diperiksa	Salah
Konvolusi	288	0

Maxpooling	147	0
ReLU	147	0
Output 8 bit	147	0

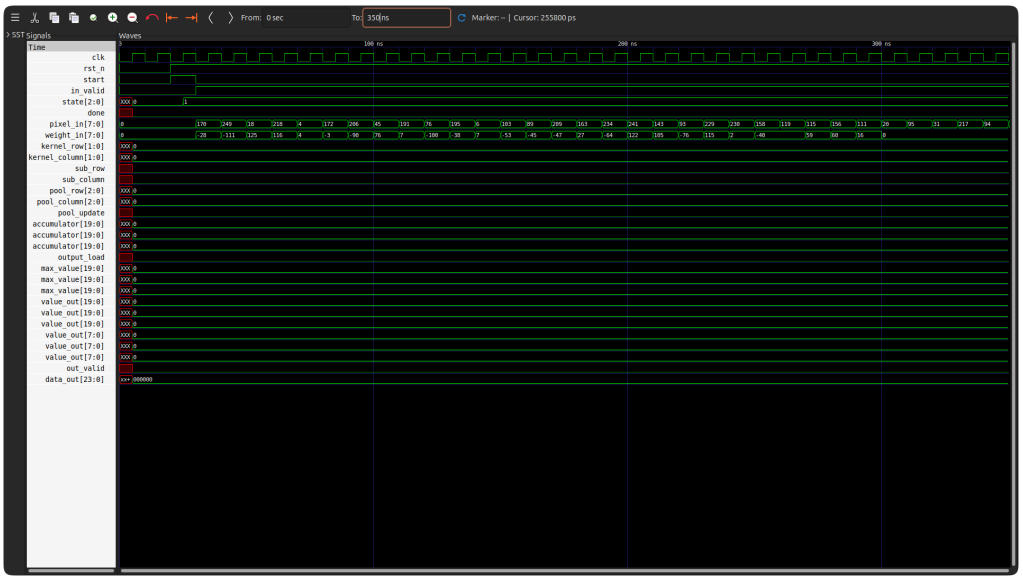
Total clock pemrosesan adalah 2265. Hasil matriks lengkap setiap tahap dapat dilihat di results/sim.log

4.2. Waveform

Siklus dihitung dari tepi clock saat start terbaca. Berikut merupakan hasil waveform

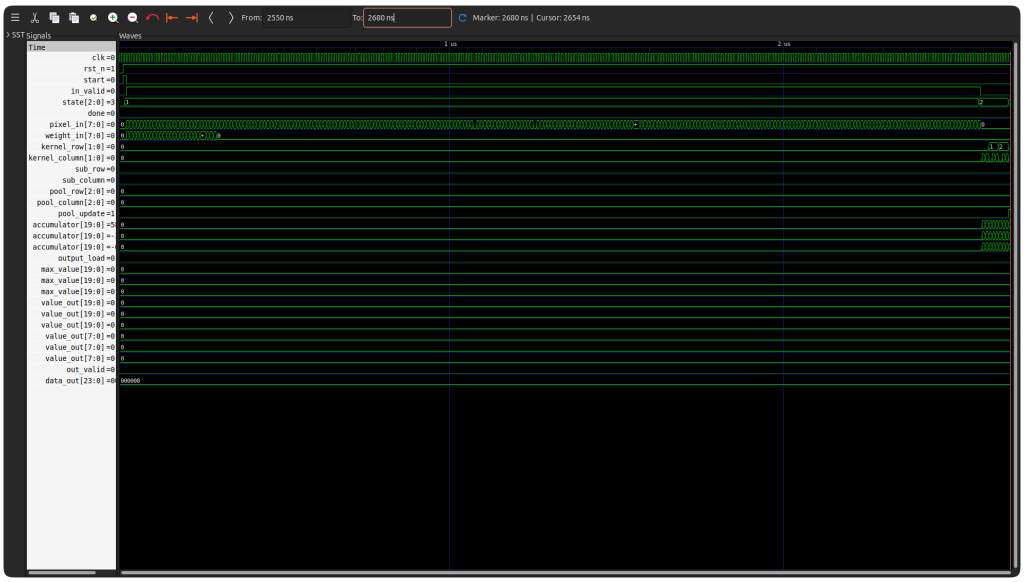


Gambar 4.2 Waveform keseluruhan akselerator

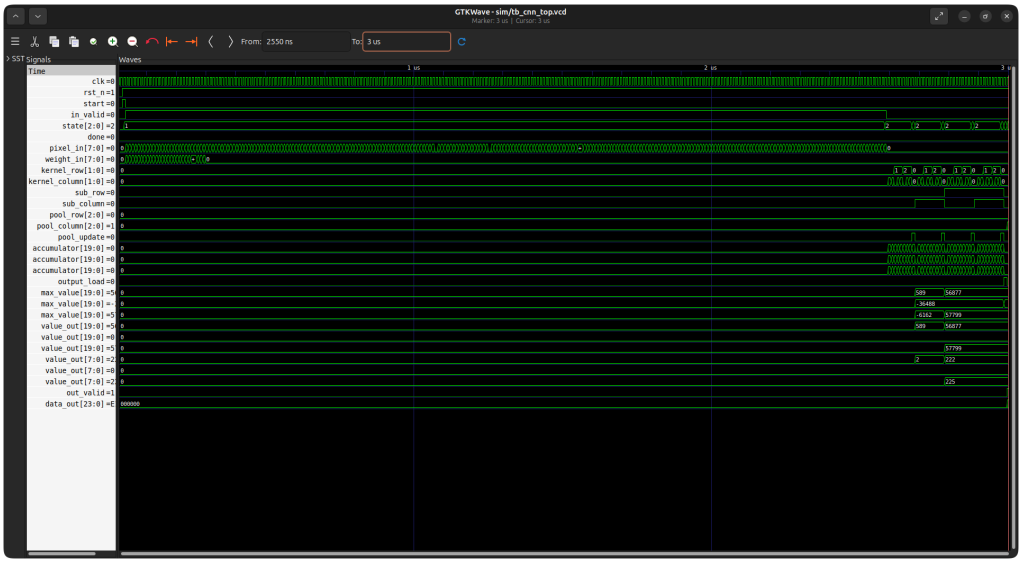


Gambar 4.3 Bagian LOAD

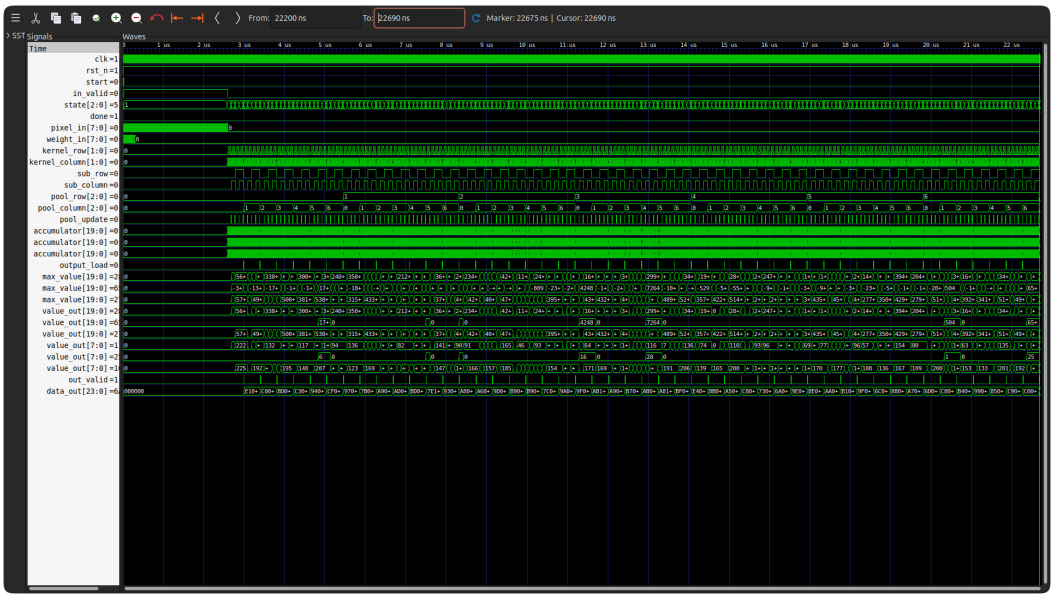
Pada clock 2, sinyal start terbaca dan state berpindah dari IDLE Ke LOAD. Selama clock 3 hingga 258, in_valid bernilai 1 dan pixel_in berganti setiap clock. Setelah seluruh data terload, last_load bernilai 1 sehingga state berpindah ke MAC



Gambar 4.3 Bagian Konvolusi



Gambar 4.1 Satu jendela pooling hingga output



Gambar 4.5 Output terakhir dan DONE

Bab V : Analisis Hasil

Desain arsitektur memiliki clock 100 MHz dan menunjukkan hasil yang identik dengan nilai referensi atau golden model seperti pada penjelasan sebelumnya. Selain itu, Desain ini juga dilakukan sintesis menggunakan Yosys, hasil sintesis dapat dilihat di results/synth_stat.txt. Berikut merupakan ringkasan hasil sintesis

Modul	Jumlah Unit	Sel per unit	Total Sel	Persentase (%)
image_buffer	1	4648	4648	59,5
conv_mac	3	575	1725	22,1
kernel_buffer	1	788	788	10,1
maxpool_unit	3	121	363	4,6
cnn_control	1	171	171	2,2
relu_unit	3	19	57	0,7
requant_unit	3	11	33	0,4
Register output (cnn_top)	1	25	25	0,3
Total			7810	100

Pada hasil sintesis dapat dilihat bahwa buffer input dan kernel memakan 69,6% sel. Ketiga MAC hanya 22,1%, FSM dan semua counter hanya 2,2%.

Bab V : Kesimpulan

Implementasi akselerator desain ini mendapatkan kesimpulan:

1. Akselerator Convolution dengan Maxpooling, ReLU dengan 3 kernel 3x3 berhasil diimplementasikan dalam RTL
2. Hasil simulasi identitk dengan golden model pada keempat tahap.
3. Sintesis menghasilkan 7810 sel dengan penyimpanan data sebagai komponen terbesar

Lampiran

Folder atau file	Isi
rtl/	8 file RTL: cnn_top.v, cnn_control.v, image_buffer.v, kernel_buffer.v, conv_mac.v, maxpool_unit.v, relu_unit.v, requant_unit.v
tb/tb_cnn_top.v	Testbench
tb/vectors/	Vektor uji dan acuan dari golden model (6 file hex)
scripts/golden_model.py	Golden model Python
results/	sim.log, tb_cnn_top.vcd, cnn_waveform.gtkw, screenshot waveform, synth_stat.txt, golden_output.txt
Makefile	Perintah simulasi, lint, dan sintesis

Perangkat yang digunakan: Icarus Verilog, GTKWave, Verilator (lint), Yosys (sintesis), dan Python 3. Berikut merupakan perintah dari Makefile:

```
make golden         # menghasilkan vektor uji dari golden model
make sim            # simulasi dan pemeriksaan, log ke sim/sim.log
make wave           # membuka waveform di GTKWave
make lint           # pemeriksaan RTL dengan Verilator
make synth              # sintesis Yosys, statistik ke sim/synth_stat.txt
```